

Premise Selection over ZFC by Learning to Rank

Search Effort as the Governing Metric,
with Case Studies in Metamath’s `set.mm`

Brian Tenneson
M.S., Applied Data Science
Currently Unaffiliated
btenneson2301@baypath.edu

Version 4.1 • July 28, 2026

Abstract

An automated theorem prover working inside a large formal library spends most of its effort deciding which of tens of thousands of established results bear on the goal in front of it. We treat that decision as a ranking problem over Metamath’s `set.mm`, a formalisation of ZFC containing on the order of 43,900 proved theorems, and make three contributions.

First, a metric. Premise selectors are conventionally scored by recall at a fixed cutoff, which measures the head of the ranking and is silent about its tail. We argue that the governing quantity for a prover is *search effort*: the fraction of the candidate pool that must be examined before every premise of a proof has been found, since that is the point at which search may stop. Recall and effort are independent — a ranker can excel at one and fail at the other — so both are reported throughout, against two baselines that a learned model must beat to justify itself.

Second, a method. We fit a linear scorer by a pairwise ranking loss over candidate pairs drawn from a single goal, rather than by classifying (goal, candidate) pairs independently. Since every reported quantity depends only on the order induced within one goal, this aligns the training objective with the measurement. It also has a structural consequence, isolated as Proposition 4.2: features that are constant across a goal’s candidates cancel identically and cannot receive weight, so a class of silently uninformative feature is excluded by construction rather than by inspection.

Third, evidence. On the first 8,000 statements of `set.mm` under a temporal split, the ranker attains recall@10 of 0.232 and recall@50 of 0.491 against 0.174 and 0.233 for the strongest baseline, and reduces search effort from 0.926 to 0.387 — from reading 2,779 of 3,000 candidates to reading 1,161. Four worked case studies show where the gain comes from and where it does not: premises sharing notation with the goal are retrieved at ranks 1 to 6 out of several thousand, while premises stated in notation absent from the goal are retrieved in the hundreds. We identify that asymmetry as the principal obstacle to further progress and propose co-citation over the premise graph as the natural remedy.

Keywords: premise selection, learning to rank, automated theorem proving, Metamath, ZFC, search effort, proof libraries, RankNet, co-citation.

Contents

1	Introduction	3
1.1	Contributions	3
2	The corpus and the premise relation	3
2.1	Metamath	3
2.2	Why the typecodes must be separated	4
2.3	Corpus statistics	4
3	Metrics	4
4	Method	5
4.1	Ranking rather than classification	5
4.2	Features	6
4.3	Negative sampling	6
4.4	Protocol	7
5	Results	7
6	Case studies	7
7	Co-citation	8
8	Related work	9
9	Limitations and further work	9
10	Conclusion	9
	Reproducibility	10
	Acknowledgements	10

1 Introduction

A theorem prover operating on a small axiom set can afford to search exhaustively. One operating inside a formal library cannot. Modern libraries — Metamath’s `set.mm`, Isabelle’s Archive of Formal Proofs, Lean’s Mathlib — contain tens or hundreds of thousands of established results, and the dominant cost of proving a new statement is not applying an inference rule but deciding *which* of those results to apply it to. This is the premise selection problem, and it is why interactive provers ship with tools such as Sledgehammer [3] whose function is precisely to guess a short list of relevant lemmas.

This paper studies premise selection over ZFC as formalised in `set.mm` [2]. The choice of corpus is deliberate and is discussed in Section 2: a Metamath proof is literally a list of label references, so the premise relation is recorded in the file rather than inferred, and no proof reconstruction or natural-language processing intervenes between the data and the learning problem.

1.1 Contributions

1. **Search effort as the governing metric** (Section 3). We define effort as the depth to which a ranking must be read before every true premise has been seen, argue that this rather than recall is what a prover experiences, and show that the two quantities diverge sharply in practice.
2. **A ranking formulation** (Section 4). We fit the scorer by a pairwise loss over candidates of a common goal. Proposition 4.2 records a structural benefit: goal-constant features cannot absorb weight.
3. **Typecode separation as a preprocessing requirement** (Section 2). In Metamath, references to syntax constructors outnumber references to mathematics by a factor approaching one in two. We quantify this and show that failing to separate them changes both what the data look like and what a model learns.
4. **Empirical results with worked case studies** (Sections 5–6). Aggregate figures on `set.mm` are accompanied by four theorems examined premise by premise, which localise the method’s strength and its failure mode more precisely than averages can.

2 The corpus and the premise relation

2.1 Metamath

A Metamath database is a sequence of statements. Each is introduced by a label and asserts a string of tokens, the first of which is its *typecode*. Two kinds concern us. A statement whose typecode is `|-` asserts provability; one whose typecode is `wff`, `class` or `setvar` asserts that a string is well formed. Axioms are marked `$a` and proved statements `$p`, and a proved statement carries its proof as a list of the labels it invokes.

Definition 2.1 (Premise relation). Let $C = (t_0, \dots, t_{N-1})$ be a Metamath database in file order. For a proved statement t_i let $R(t_i)$ be the set of labels its proof references. The *premises* of t_i are

$$P(t_i) = \{t_j : j < i, \text{label}(t_j) \in R(t_i), \text{typecode}(t_j) = |- \}.$$

The remaining references, those to statements of syntactic typecode, are the *syntactic references* of t_i .

2.2 Why the typecodes must be separated

Definition 2.1 restricts to logical typecode, and this restriction is not cosmetic.

Observation 2.2 (Syntactic references dominate). In the first 8,000 statements of `set.mm`, of 83,923 total references 35,902 — 42.8% — are to statements of syntactic typecode. Counting them as premises raises the mean number of premises per proof from 6.2 to 10.8.

The reason is structural rather than incidental. Metamath requires a proof to construct its formulas before asserting anything about them, so the constructors for implication, conjunction, membership and equality are invoked by essentially every proof in the library, independently of its subject. Table 1 makes the effect visible: computed over all references, the most-cited labels are uniformly grammatical; computed over logical references only, they are the inference lemmas one would expect.

all references			logical references only		
label	role	count	label	role	count
wcel	syntax: $A \in B$ is a wff	2946	syl	sylogism	775
wa	syntax: conjunction	2790	bitri	biconditional transitivity	641
wceq	syntax: equality	2545	ax-mp	modus ponens	482
cv	syntax: setvar as class	2365	ali	antecedent introduction	348
wi	syntax: implication	1814	ex	exportation	342
wn	syntax: negation	1314	vex	setvar existence	322

Table 1: Most-referenced labels in the first 8,000 statements of `set.mm`. Without typecode separation the premise relation is dominated by grammar; a model fitted to it learns to predict punctuation.

Remark 2.3 (Generality). The same distinction arises in any system separating syntax from assertion. In Isabelle and Lean it is largely handled by the elaborator and is invisible in the proof term; in Metamath it is explicit in the file, which is convenient for the present purpose but requires the filter of Definition 2.1.

2.3 Corpus statistics

After separation, the first 8,000 statements comprise 7,899 logical statements and 101 syntactic ones, with 48,021 logical references. Proofs cite a mean of 6.2 premises, median 4, maximum 101; the distribution of citations is heavily skewed, with a small number of inference lemmas appearing in hundreds of proofs and the majority of statements cited once or twice.

3 Metrics

Fix a goal g with true premise set G , and a candidate pool Π of statements available to it. A selector induces a total order ρ on Π .

Definition 3.1 (Recall). $\text{recall}@k = \frac{|G \cap \{\rho_1, \dots, \rho_k\}|}{|G|}$.

Definition 3.2 (Search effort). $\text{effort} = \frac{\max\{r : \rho_r \in G\}}{|\Pi|}$, the depth to which the ranking must be read before every premise has been seen.

Remark 3.3 (Why effort is the governing quantity). Recall@ k describes the head of a ranking. It is the right measure for a system that will be handed a fixed-size shortlist and must do the best it can with it. It is the wrong measure for a prover that needs *all* the premises of a proof, because such a prover cannot stop at k : if one premise lies at rank 2,000, a ranking that placed the other four at ranks 1 to 4 has saved it nothing. Effort measures exactly the stopping point.

The two are independent. A ranking with four premises at ranks 1–4 and the fifth at 2,000 has excellent recall@5 and effort near 1; a ranking with all five at ranks 40–50 has poor recall@5 and good effort. Neither implies the other, and Section 5 exhibits both patterns in the same experiment.

Definition 3.4 (Baselines). Two orderings serve as controls. *Frequency* sorts candidates by how often each has been cited in the training prefix. *Chronological* sorts by recency, most recently proved first; since a library is developed topically, a goal’s premises are frequently among the statements proved shortly before it, and reading the library backwards is a strong and entirely unlearned strategy. We treat chronological order as the brute-force benchmark.

Remark 3.5 (Both metrics against both baselines). A single number invites a misleading summary in either direction. Reporting effort alone flatters a selector that returns a short list and misses premises; reporting recall alone flatters one that finds the easy premises and buries a hard one. We therefore report both, against both baselines, throughout.

4 Method

4.1 Ranking rather than classification

The natural first formulation treats premise selection as binary classification: fit $\Pr(\text{premise} \mid g, c)$ over (goal, candidate) pairs and sort candidates by the fitted probability. We adopt a ranking formulation instead, for a reason visible in Definitions 3.1 and 3.2: every reported quantity is a function of the order induced *within a single goal*. Absolute probabilities are never compared across goals and never consulted; only their relative order within a goal is.

Definition 4.1 (Pairwise ranking loss). Let g be a goal, $p \in P(g)$ a premise, $n \notin P(g)$ a non-premise available to g , and $x_p, x_n \in \mathbf{R}^d$ their feature vectors. The scorer is linear, $s(x) = w \cdot x$. We fit w by logistic regression on difference vectors,

$$\sigma(w \cdot (x_p - x_n)) \rightarrow 1, \quad \sigma(w \cdot (x_n - x_p)) \rightarrow 0,$$

over all pairs (p, n) drawn from a common goal, where $\sigma(z) = (1 + e^{-z})^{-1}$. This is RankNet [1] with a linear scorer; the antisymmetric second family of examples prevents the objective from being satisfied by a uniform inflation of scores.

Proposition 4.2 (Goal-constant features cannot receive weight). *Suppose a feature coordinate j satisfies $x_c^{(j)} = \kappa_g$ for every candidate c available to a goal g , with κ_g depending on g alone. Then the j th coordinate of every difference vector formed from g is zero, the partial derivative of the loss with respect to w_j vanishes on every such example, and w_j is unaffected by the data.*

Proof. For any pair (p, n) of candidates of g we have $x_p^{(j)} - x_n^{(j)} = \kappa_g - \kappa_g = 0$. The loss depends on w only through the inner products $w \cdot (x_p - x_n)$, whose derivative with respect to w_j is the j th coordinate of the difference vector, namely zero. Summing over examples, $\partial \mathcal{L} / \partial w_j = 0$ identically. $\square$

Remark 4.3 (Consequence for feature design). Proposition 4.2 is a modest observation with a practical use. A feature depending only on the goal — the goal’s length, its depth in the library, the number of its premises — is uninformative for ranking, since it shifts every candidate’s score by the same amount. Under a classification objective such a feature can nonetheless acquire a large coefficient, because it genuinely predicts the class marginal, and will appear informative in a table of learned weights. Under Definition 4.1 it is excluded by construction. The point generalises: the ranking formulation restricts the model to exactly the information that can affect the reported metrics.

4.2 Features

All features are properties of a (goal, candidate) pair and are computable from the database without proof reconstruction. Writing S_g, S_c for the token sets of goal and candidate:

1. *Symbol overlap*, $|S_g \cap S_c| / |S_g \cup S_c|$, and the two asymmetric variants $|S_g \cap S_c| / |S_c|$ and $|S_g \cap S_c| / |S_g|$.
2. *Literal occurrence*: whether the candidate’s statement occurs as a substring of the goal’s.
3. *Citation frequency*: $\log(1 + u_c)$ where u_c counts citations of c in the reference prefix.
4. *Recency*: the positional gap between goal and candidate.
5. *Size*: the candidate’s length, and its ratio to the goal’s.
6. *Rare-symbol sharing*: whether goal and candidate share a token in the lowest quartile of corpus frequency.
7. *Local co-citation*: how often the candidate is cited by the 40 statements immediately preceding the goal.
8. *Co-citation with anchors*: see Section 7.

4.3 Negative sampling

Training pairs require non-premises, and their selection is not neutral. Uniform sampling from the available prefix supplies negatives that are almost all trivially irrelevant — on a corpus of tens of thousands, a random statement bears no relation to a given goal — so a model trained against them learns to reject the unrelated rather than to discriminate among the plausible, which is the operative task.

We therefore draw a majority of negatives from the most-cited statements of the reference prefix: lemmas that are plausible for any goal because they are used everywhere, and which the model must learn to rank *below* the goal’s actual premises.

Remark 4.4 (Negatives must be selected orthogonally to the features). A design constraint deserves emphasis, as it is easy to violate. If negatives are sampled by a criterion that some feature also measures, that feature becomes a detector for the sampling procedure. Concretely: drawing hard negatives from the premises of the 40 statements preceding the goal, while also supplying a feature counting citations within that same window, makes the feature predict membership in the negative class almost perfectly — and a correctly fitted model will assign it a large negative weight, degrading performance on genuine data where no such relationship holds. Selecting negatives by global citation frequency, a quantity no locality feature computes, avoids the coupling.

4.4 Protocol

Definition 4.5 (Temporal split). Given a corpus in derivation order and a fraction p , the first $\lfloor pN \rfloor$ statements supply training goals and the remainder supply test goals. A random split would allow the model to train on statements postdating its test goals, which does not correspond to any deployment condition.

Definition 4.6 (Reference prefix). Several features — citation frequency, symbol rarity, co-citation — are defined relative to a corpus. The prefix over which these are computed, the *reference prefix*, is specified separately from the training cut and held fixed across any experiment that varies the amount of training data, so that a feature denotes the same quantity during fitting and scoring.

5 Results

We report on the first 8,000 statements of `set.mm`, temporal split at $p = 0.8$, 200 held-out goals, candidate pools of 3,000.

	ranker	frequency	chronological	fused
recall@1	0.051	0.036	0.085	0.053
recall@5	0.155	0.092	0.146	0.123
recall@10	0.232	0.131	0.174	0.168
recall@50	0.491	0.279	0.233	0.476
effort	0.387	—	0.926	0.321
candidates read (of 3000)	1161	—	2779	964

Table 2: Aggregate performance on `set.mm`. The fused column is reciprocal-rank fusion of the learned ranking with the frequency ordering.

Observation 5.1 (The baseline wins at the head). Chronological order attains the best recall@1, at 0.085 against the ranker’s 0.051. A library’s most recently proved statement is a strong single guess, because development proceeds topically and a new theorem commonly uses the lemma established immediately before it. No learned model in our experiments displaced this baseline at $k = 1$.

Observation 5.2 (The learned ranker wins everywhere else). From $k = 5$ upward the ordering reverses and the margin widens: at $k = 50$ the ranker recovers 0.491 of premises against 0.233 for chronological order, a factor of 2.1. On effort the difference is starker still: the ranker requires 1,161 of 3,000 candidates against 2,779, and rank fusion with the frequency ordering reduces this to 964.

Remark 5.3 (Reading the two together). Observations 5.1 and 5.2 are the empirical content of Remark 3.3. Chronological order is better at its first guess and much worse at completing the set. A system permitted one attempt should read the library backwards; a system that must assemble every premise of a proof should use the learned ranking, and the difference between them — roughly 1,600 candidates of reading — is the practical contribution.

6 Case studies

Aggregate figures conceal structure. We examine four theorems of `set.mm` premise by premise; in each the candidate pool is every logical statement preceding the goal, several thousand in size.

theorem	statement	ranker	frequency	chronological
prcom	$\{A, B\} = \{B, A\}$	0.080	0.978	0.404
uncom	$(A \cup B) = (B \cup A)$	0.077	1.000	0.932
unass	$((A \cup B) \cup C) = (A \cup (B \cup C))$	0.285	0.997	0.933
elpri	$A \in \{B, C\} \rightarrow (A = B \vee A = C)$	0.135	1.000	0.942

Table 3: Search effort on four theorems. Lower is better; the ranker improves on the better baseline by factors of 3.3 to 12.1.

Example 6.1 (Commutativity of unordered pairs). The statement $\{A, B\} = \{B, A\}$ has a three-premise proof: **df-pr**, that $\{A, B\} = \{A\} \cup \{B\}$; **uncom**, that union commutes; and **3eqtr4i**, chaining three equalities. The argument is $\{A, B\} = \{A\} \cup \{B\} = \{B\} \cup \{A\} = \{B, A\}$.

Out of 4,643 candidates the ranker places **df-pr** at rank 2, **3eqtr4i** at 150 and **uncom** at 372; frequency places them at 4541, 1036 and 4067. The definitional premise is found almost immediately, because the goal contains the literal tokens $\{A, B\} =$ and so does **df-pr**.

Observation 6.2 (Where the method succeeds). Across the four theorems, premises sharing notation with the goal are retrieved at ranks 1 to 6: **uneqri** at 1 and **elun** at 3 for **uncom**; **uneqri** at 4 and **elun** at 5 for **unass**; **elprg** at 6 for **elpri**; **df-pr** at 2 for **prcom**. Symbol overlap is close to decisive in this regime.

Observation 6.3 (Where it fails). Two classes of premise are retrieved poorly. The first is generic inference lemmas — **orcom** at rank 312 against frequency’s 62, **orbi1i** at 933 against 215, **3bitrri** at 1163 against 307. These appear in proofs across all of mathematics and share no vocabulary with any particular goal, so citation frequency predicts them and overlap cannot.

The second is more fundamental. In Example 6.1 the premise **uncom** is ranked 372: the proof routes through union, but the goal $\{A, B\} = \{B, A\}$ contains no union symbol. *Any* feature computed from shared vocabulary is structurally incapable of retrieving a premise whose notation does not appear in the goal.

Remark 6.4 (The two failure modes are different). The first is addressable by combination: frequency and the learned ranker fail on disjoint sets of premises, and Table 2 shows that fusing them improves aggregate effort from 0.387 to 0.321. The second is not, because no amount of reweighting recovers information the features never encoded.

7 Co-citation

Observation 6.3 identifies the obstacle: a premise stated in notation absent from the goal is invisible to vocabulary-based features. We propose a feature that measures a lemma’s company rather than its vocabulary.

Definition 7.1 (Co-citation). For statements a, b let $c(a, b)$ be the number of proofs in the reference prefix citing both. Given a goal, let A be a small set of *anchors* — candidates the vocabulary features already rank highly — and define the co-citation score of a candidate u as $\sum_{a \in A} c(u, a)$.

Remark 7.2 (Why this can reach what overlap cannot). Return to Example 6.1. The goal’s notation points unambiguously at **df-pr**, which the ranker places at position 2; **df-pr** therefore serves as an anchor. If **uncom** is cited alongside **df-pr** throughout **set.mm** — as it is, since unfolding a pair into

a union is the standard route to any statement about pairs — then `uncom` receives a high co-citation score despite sharing no symbol with the goal. The mechanism is that of collaborative filtering: relatedness is inferred from co-occurrence rather than from content.

8 Related work

Premise selection has been studied principally in the Isabelle and Mizar ecosystems, where Sledgehammer’s relevance filter [3] established the practical value of restricting a goal’s premise set before invoking an external prover. Learning-based selectors followed, and surveys of the modern neural literature are available [4]. Our contribution is not a stronger model — the scorer here is linear, and deliberately so — but a reframing of the evaluation and of the training objective, together with an analysis localising where surface features suffice and where they cannot.

The pairwise ranking formulation is RankNet [1], standard in information retrieval and, so far as we are aware, less commonly applied to premise selection than pointwise classification. The metric we advocate, search effort, plays the role that rank-of-last-relevant-item plays in retrieval settings where a user must find every relevant document rather than any.

9 Limitations and further work

Scale. Results are reported on the first 8,000 statements of `set.mm`, roughly a fifth of the corpus. Whether the margin over the baselines is preserved, widened or eroded as the library grows is open, and the question is sharpened by the heterogeneity of `set.mm`: its early portion is propositional logic and its later portion set theory, and a single linear scorer may not serve both.

Model class. The scorer is linear. A tree ensemble can represent interactions between features — that vocabulary overlap should count for more on a content lemma than on a generic inference rule — which a linear model cannot. We regard this as the most promising direction for improving the head of the ranking, where Observation 5.1 shows a baseline still winning.

Structural features. All features here are computed from token sets. Genuine structural matching — testing whether a candidate’s conclusion unifies with a subterm of the goal modulo substitution — would address Observation 6.3 directly rather than circumstantially, at substantially greater computational cost.

Harder targets. The theorems examined in Section 6 have shallow proofs that unfold a definition and apply a propositional lemma. Cantor’s theorem, that no function maps a set onto its power set, is the natural next test: its proof constructs the diagonal set $\{y \in x : y \notin f(y)\}$ and derives a contradiction, so its premises concern separation, function application and negation — machinery whose notation need not appear in the statement at all. It is precisely the regime of Observation 6.3.

10 Conclusion

Premise selection is a ranking problem, and it is worth measuring and training as one. Measuring it as one means reporting search effort alongside recall, since a prover that must assemble every

premise of a proof is governed by the depth of its worst-ranked premise and not by the quality of its top few guesses. Training it as one means fitting the scorer to the order induced within a goal, which matches the objective to the measurement and, by Proposition 4.2, excludes a class of uninformative feature by construction.

On ZFC as formalised in `set.mm` the resulting selector reads 1,161 of 3,000 candidates where reading the library in order requires 2,779, and retrieves premises sharing notation with the goal at ranks 1 to 6 out of several thousand. It does not retrieve premises stated in unfamiliar notation, and the case studies show why: no feature computed from shared vocabulary can. Whether co-citation over the premise graph closes that gap is the question we regard as most worth answering next.

Reproducibility

All results were produced by a single self-contained Python program depending on no third-party libraries. Every run writes a timestamped directory containing the fitted model, the measured quantities, and the corpus statistics used.

```
python predator4.py fetch set.mm
python predator4.py stats --db set.mm --limit 8000
python predator4.py train --db set.mm --limit 8000 -p 0.8
python predator4.py prove --db set.mm --label prcom --limit 12000
```

Acknowledgements

The author thanks AI assistants used as engineering and verification tools during the preparation of this work, including for code review, verification of cross-references, and the identification of several methodological faults during development. All mathematical claims, interpretations and final decisions remain the responsibility of the author.

References

- [1] Burges, C., Shaked, T., Renshaw, E., Lazier, A., Deeds, M., Hamilton, N., & Hullender, G. (2005). Learning to rank using gradient descent. *Proceedings of the 22nd International Conference on Machine Learning*, 89–96.
- [2] McGill, N. D., & Wheeler, D. A. (2019). *Metamath: A Computer Language for Mathematical Proofs* (2nd ed.). Lulu Press.
- [3] Paulson, L. C., & Blanchette, J. C. (2010). Three years of experience with Sledgehammer, a practical link between automatic and interactive theorem provers. *Proceedings of IWIL*.
- [4] Li, Z., Sun, J., Murphy, L., Su, Q., Li, Z., Zhang, X., Yang, K., & Si, X. (2024). A survey on deep learning for theorem proving. *Conference on Language Modeling*.
- [5] Tenneson, B. (2026). *Depths of a Simulation and Formalized Self-Awareness*, Merged Edition.